智能体驱动的 AI
安全知识情报系统设计与实现

技术报告

目录

目录 1

作品概览 1

核心亮点 1

关键数字（2026-10-13 快照，取演示机数据库） 2

如实说明的边界 2

一、系统总体架构设计 2

二、智能体方案设计 3

三、情报监测模块详细设计 4

四、情报富化模块详细设计 5

五、情报问答模块详细设计 6

六、关键技术选型说明 6

七、系统工作流程图 6

八、系统实现细节与核心代码说明 7

附录 A：术语与证据索引 8

九、各功能模块实现效果展示 8

十、性能测试结果与分析 9

十一、创新点总结 10

附录 B：时效性的混合口径（保留，不进正表） 10

附：评分细则档位对照 11

作品概览

本系统面向 AI 安全漏洞情报这一具体问题：把分散在 CVE/NVD、安全社区、厂商公告、安全博客、学术论文与标准政策中的 AI 安全信息自动汇聚成可检索、可追溯、可处置的知识底座，并在其上提供带引用、会拒答的自然语言问答。系统已在本机连续运行，所有指标都能从接口回读。

核心亮点

- 多源自动采集：登记 20 个来源、覆盖 11 个类别，实际产出 15 个来源；无人值守集合按小时间隔增量轮询。
- 六维富化：CVSS、受影响版本区间、修复版本、论文关联、POC 引用、资产研判，每个字段都带来源标识。
- 证据约束问答：答案只从本地证据拼装并携带引用（chunk_id + 字符偏移），资料不足时拒答而不是编造。
- 可执行的自愈：失败分类 → 有限重试 → 连续失败降级或切换备用源 → 告警，并有可复现的演练证据。
- 工程化交付：容器九项验收、CI 三个 job、回滚演练、结构化日志与告警 webhook。

关键数字（2026-10-13 快照，取演示机数据库）

项目	数值	口径
语料规模	637 个事件 / 637 个富化样本	演示机 data/intel.sqlite，快照 2026-10-14
连续监测	累计 20 个采集日，定时运行 10 天	赛题连续运行只认 trigger=scheduled
监测时效性	p50 = 15.69 小时；6 小时内 31.8%	296 个有效样本，已排除监测前历史回填
富化准确率	关系 Precision 1.0 / Recall 0.734	抽样穷尽金标准，94 条可判定样本
问答命中	引用命中 0.9722 / 预期词命中 0.9211	50 题正式评测集，2026-10-12 批次
处置闭环	高优先级 53 / 已关闭 4（7.55%）	仅计入系统复测通过的条目
自动化测试	701 passed, 0 skipped, 0 failed	交付机全量 pytest（快照 2026-10-14）
容器验收	9 项检查全部通过	reports/docker-validation.json

如实说明的边界

为避免把缺口说成能力，以下五项在正文中同样保留：没有实时监测（全部来源 realtime=false）；监测时效性 p50 为 15.69 小时，只达到赛题合格档；跨文档路径 10/21，缺 52 条文档间边；POC
与资产两个维度在评测语料中无数据；**连续运行目标在当前窗口未达成**：计划任务窗口 2026-10-11 到期后 10-12 无计划运行，连续天数已重新起算，需恢复运行后重新累积 7 天。关系维度的 1.0 / 0.734 只代表抽样范围，不外推为全语料指标。

一、系统总体架构设计

系统由输入、编排、工具、知识四层组成，四层是同一任务的不同视角，不是四套互相割裂的功能。输入层把一句自然语言任务转换成明确的目标与可校验约束，且不接受任意采集地址，数据源只能从登记列表中选择。编排层按固定流程组织采集、归一、补证、研判与回答，并把步骤状态、角色、预算与耗时全部落库。工具层只调用已登记的数据源与本地确定性能力，错误、跳过与部分成功都不得被隐藏。知识层保存事件、证据、资产、关系与结论，并要求关系必须带证据

ID，未知显示未知。

图 1 系统总体架构：四层职责与数据流

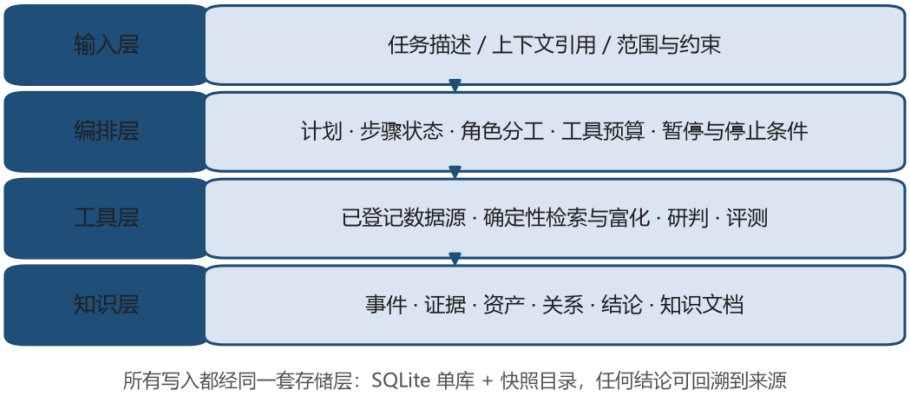


图 1 系统总体架构：四层职责与数据流

分层带来一个具体好处：任何一层的失败都不会被上一层粉饰。采集层失败时状态记为 failed 或 stale，编排层如实记录部分成功，页面按持久化状态渲染，而不是按期望状态渲染。这也是后面所有指标都能被追问、能被复核的前提。

二、智能体方案设计

系统使用六个角色：planner 负责把目标拆成计划，collector 执行采集，normalizer 做归并与去重，retriever 负责检索与证据准备，auditor 做审核与缺口标注，scheduler 负责计划任务触发。角色之间通过任务 DAG 交接，节点依赖、检查点与交接记录都写入数据库，因此一次任务的每一步都能回放。

图 2 智能体编排：角色分工与交接

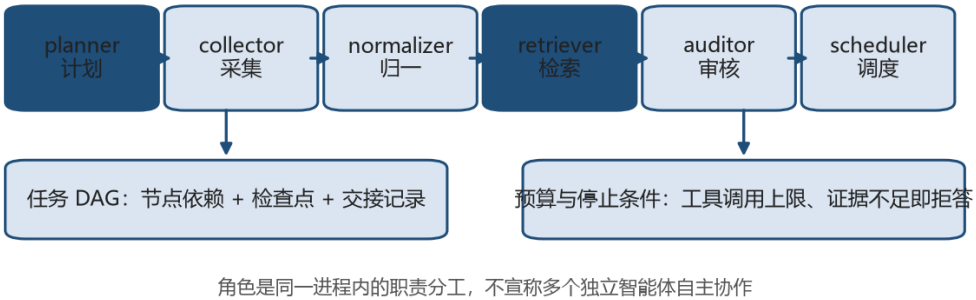


图 2 智能体编排：角色分工与交接

为什么这样编排而不是做成一个完全自主的开放式智能体：本赛题的对象是漏洞情报，结论需要能被安全从业者追溯到原始来源。开放式的自主决策会把不确定性藏进自然语言里，而固定角色 + 固定工具 + 明确预算是可以逐个环节验证的。系统同时给智能体设置了工具调用上限与停止条件，

证据不足时选择拒答，而不是继续调用工具把答案凑出来。

实际运行记录：6 个角色共产生 455 次工具调用，781 条运行记录，其中 112 次由外部计划任务触发，外部自动化已独立验证。

三、情报监测模块详细设计

采集层登记 20 个来源，分为 11 个类别，覆盖赛题要求的 CVE/NVD、厂商公告、安全社区、安全博客、学术论文、标准政策与事件库。每个来源是一条登记项，包含采集方式（api / feed / rss / page）、独立来源标记、可信度与许可说明；客户端无法提交任意 URL。

数据源类型	具体数据源	更新频率	实际产出
CVE/NVD	nvd、mitre_cve	1 小时增量	273 / 202 条
开源生态漏洞库	osv	1 小时增量	91 条
安全社区与聚合公告	ghsa	1 小时增量（需只读令牌）	108 条
在野利用情报	cisa_kev	1 小时增量	26 条
厂商安全公告	msrc	1 小时增量	10 条
安全博客与社区	freebuf、anquanke、talos_blog 等 7 个	1 小时增量	共 35 条
学术论文	arxiv、openalex	openalex 1 小时；arxiv 按需	114 / 36 条
标准政策与框架	eu_ai_act、nist_ai_rmf、owasp_genai 等	1 小时增量	共 7 条

更新频率只有一档：推荐集统一小时间隔增量轮询，机器休眠时由计划任务的补跑策略顺延。系统没有按来源分档频率的字段，因此不会在报告里写成核心源与全量分层轮询。需要特别说明的是实时监测：全部来源 realtime 均为 false，本系统是小时级，不是实时系统。

来源状态只有五种取值，并且都能在接口里看到：ok 表示本轮成功；failed 表示本轮失败且没有历史数据；stale 表示本轮失败但历史数据仍在，因此检索不受影响、只是没有新数据；partial 表示部分成功，系统会把未处理完的部分留到下一轮而不是标记成功；skipped 表示主动跳过，例如缺少只读令牌的 GHSA。把这五种状态区分开，是为了让“ 部分成功 ”不被读成“ 成功 ”。

图 5 采集状态机：五种状态的定义与迁移

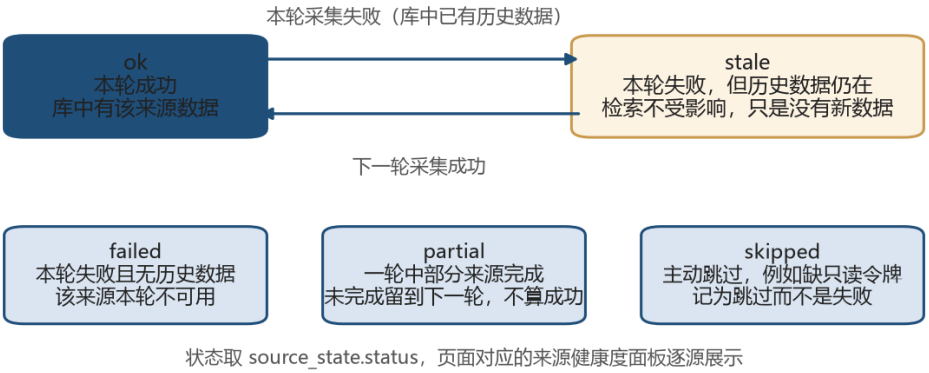


图 5 采集状态机：五种状态的定义与迁移

重复信息在采集后会进入去重候选表（当前 210 条候选），同一漏洞的多来源记录不会各自成为独立知识点，而是合并成一个事件并保留各自的来源标识。这样问答引用到的是一条事件，但答题时仍能指出这条结论来自 NVD 还是 OSV。

四、情报富化模块详细设计

富化对采集到的事件做多维扩展。六个维度在演示机上都有数据：CVSS 覆盖 388 个事件、论文关联 150 个、修复版本 101 个、POC 引用 74 个、资产研判 48 个、带证据关系 26 个。每个字段都保存来源标识，可以逐条回读原始快照。

关联推理的关键约束是不合并证据。以真实事件 GHSA-w794-rj3p-xv45（Langflow 命令注入）为例，同一事件下有三条受影响区间，分别属于 1.x 与 0.x 两条版本线，各自带独立 source_id 与各自的修复版本。系统没有把它们合并成一条更整洁的结论，因为合并会让 0.x 线的用户拿到错误的修复版本。完整案例见 docs/CASE_ENRICHMENT_INFERENCE.md。

必须如实说明的是 POC 与资产两个维度的边界：在 B 评测语料中 assets、assessments 两张表无记录，events[].poc[] 也为空，因此这两个维度没有可计算的分母；报告里写无数据，不写 0。演示机上的资产条目为合成资产，结论限定为策略感知处置建议，不声称个性化资产推荐。

富化维度	演示机覆盖	数据来源
CVSS 评分	388 / 596 事件	事件文档 cvss[]，含向量与评分
论文关联	150 / 596 事件	事件自身的论文属性或已保存的预印本来源
修复版本	101 / 596 事件	affected[].fixed_version，逐条带 source_id
POC 引用	74 / 596 事件	事件 poc[]，回填自 NVD 的 Exploit 标签引用
资产研判	48 / 596 事件	assessments 表，事件与资产的处置研判记录
带证据关系	26 / 596 事件	relationships 中至少一条关系带 evidence_ids

覆盖率的读法需要克制：覆盖率低不等于这是缺陷，例如论文关联只对论文类事件有意义。报告里给出的是“有该维度数据的事件数 / 全部事件数”，不是准确率，两者不能互相替代。

五、情报问答模块详细设计

问答链路基于本地知识库做关键词与语义混合检索，答案只从检索到的证据拼装，并携带引用。每条引用包含 chunk_id、字符起始与结束偏移，正文可以按偏移回读核对。检索层对中文短语做英文同义词扩展，以便在英文语料上工作；扩展词参与匹配但按一个概念计分，避免出现提问词压过主题词的情况。

评测集为固定 50 题，覆盖基础事实问答 10 题、术语与语义查询 8 题、中英文混合查询 6 题、多轮追问 6 题、跨文档问答 8 题、真实多跳问答 2 题，以及无证据与拒答各 5 题。2026-10-12 批次实测：引用命中 35/36 = 0.9722，预期词命中 35/38 = 0.9211，拒答一致率 42/50 = 0.84，引用总数 222。完整演示脚本见 docs/CASE_QA_MULTITURN_CROSSDOC.md。

跨文档能力需要区分两个层次：在评测集上，跨文档题能答对，因为期望文档确实存在于语料中；但在知识图谱层面，跨文档路径只通过 10/21，仍缺 52 条文档间引用边。因此本模块的定位是跨文档检索与聚合，不是完整跨文档推理。

拒答是这条链路的一等公民，不是失败兜底。评测集里 5 道无证据题与 5 道证据不足题专门考核这一点：当检索不到可用证据时，系统返回拒答并说明原因，而不是用模型的常识补齐答案。同一批 50 题的作答与拒答结构为 46 / 4，与上一批次完全一致，说明本轮检索改动没有改变拒答口径。

引用可追溯性由测试固定：每条引用都要能按 chunk_id 与字符偏移回到原始分块，偏移与存储文本不一致的记录会被契约检查拒绝。这条约束让“引用能找回”与“引用支持答案”成为两件被分别检查的事情——前者有引用一致性测试，后者由预期词命中率度量。

六、关键技术选型说明

选择	理由	代价与边界
Python + FastAPI	单进程内即可完成采集、检索、研判与问答编排，部署简单	长任务依赖计划任务而非常驻队列
SQLite 单库 + 快照目录	证据与结论同库，任何字段可回溯；备份即一个文件	并发写入受限，不适合多实例水平扩展
确定性检索（关键词 + 同义词扩展）	离线可跑、结果可复现、不依赖外部模型	语义泛化弱于向量检索，需靠扩展词弥补
模型为可选增强	未配置模型时系统仍可完整运行并给出证据	问答为证据拼装式回答，长文本生成能力有限
Docker + Compose + CI	本机与 CI 使用同一镜像定义，验收可复现	容器内不携带语料，评测语料需另行交付

七、系统工作流程图

图 3 系统工作流：采集 → 富化 → 问答 → 处置闭环

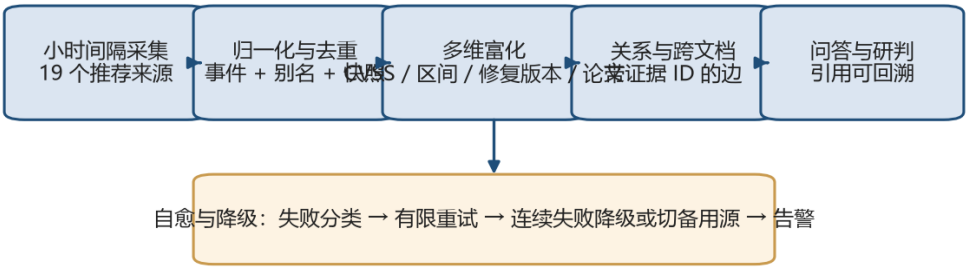


图 3 系统工作流：采集 → 富化 → 问答 → 处置闭环

计划任务每小时触发一次采集，采集结果先经过归一化与去重写入事件库，再进入富化与关系构建。问答与研判读取同一份知识库，回答带引用返回。任一环节失败都进入自愈分支：先做失败分类，再执行一次受控重试；重试仍失败则计入连续失败，达到阈值后该来源退出无人值守采集或切换到备用源，同时发出告警。整个链路的状态都落在 runs 与 source_state 两张表里，可逐条核查。

图 6 自愈状态机：失败 → 重试 → 降级 → 恢复

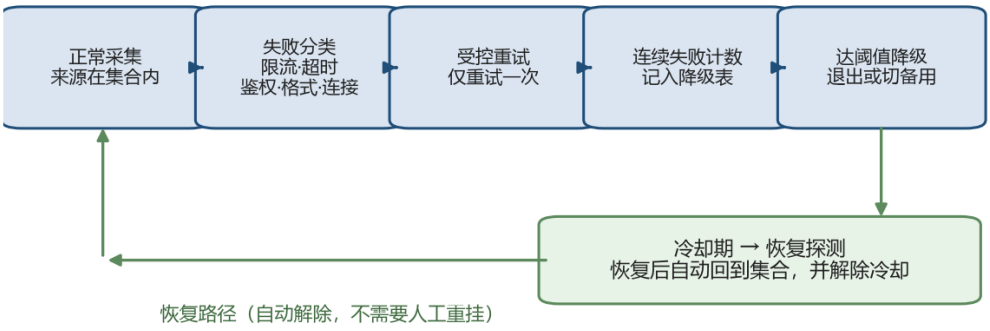


图 6 自愈状态机：失败 → 重试 → 降级 → 恢复

八、系统实现细节与核心代码说明

采集编排入口是 app/agents.py 的 run_collection：逐个来源调用采集器，把结果写回 source_state，并把每个来源的状态、抓取数、保留数、过滤数、耗时与错误写入运行记录。自愈策略在 app/self_healing.py，核心是三步：classify_failure 把错误归类为限流、超时、鉴权、格式变化、连接重置或上游拒绝；recovery_decision 决定是否值得重试；degradation_for 在连续失败达到阈值时给出降级动作。

降级状态存放在独立表 source_degradations 而不是给 source_state 加列，这样已有数据库靠 CREATE TABLE IF NOT EXISTS 就能升级，不需要迁移脚本。采集挑选来源时经过 apply_degradation_policy：处于冷却期的来源被移出，声明了备用源的来源由备用源代采，并把每一次替换写进运行记录，避免来源悄悄消失。

可复现的自愈演练在 tools/run_self_healing_drill.py：它在数据库副本上注入可控故障，运行真实的自愈代码路径，验证四项断言——故障来源退出无人值守集合、恢复后自动回到集合、无备用源的来源走降频、声明备用源的来源切换成功。结果写入 reports/self-healing-drill.json。

数据层只有两个位置：一个 SQLite 数据库保存事件、别名、来源状态、资产、研判、处置、运行记录与分块索引；一个快照目录保存原始响应。事件的每个来源引用都指向一个快照哈希，因此“这条结论从哪来”可以一路查到原始字节。备份与回滚也按这个结构设计：停服务、保留当前库、用备份覆盖、重启并校验健康接口，演练记录在 reports/backup-restore-validation.json。

问答与研判共用一套检索接口，避免出现“页面看到的证据”和“机器评判用的证据”来自两套实现。这一点直接决定了指标的可信度：页面、接口与证据文件读的是同一份持久化状态，前端不做二次换算，也不做估算。

附录 A：术语与证据索引

术语	含义	在本报告中的位置
事件	归一化后的一个漏洞或知识条目，可合并多个来源	第三章、第九章
证据 ID	指向某个快照中一段具体内容的标识	第四章、第五章
stale	本轮采集失败但历史数据仍可用，检索不受影响	第三章
降级	来源连续失败后退出无人值守采集或切换备用源	第七章、第八章
抽样穷尽	在声明的样本范围内逐条枚举应抽取关系，而非全语料穷尽	第四章、第十章
口径基线	对外引用的指标一律取演示机数据库读数	第一章、第十章

配套证据文件：docs/COMPETITION_RUBRIC_MAPPING.md（评分档位对照）、docs/METRICS_BASELINE.md（口径基线）、docs/CASE_ENRICHMENT_INFERENCE.md（富化案例）、docs/CASE_QA_MULTITURN_CROSSDOC.md（问答演示）、reports/（容器与自愈验收产物）。

九、各功能模块实现效果展示

模块	展示方式	证据文件
监测与来源健康	赛题指标页的来源健康度面板（每个来源的状态、类别、最近成功时间、产出条数、连续失败次数）	docs/screenshots/09-source-health.png、/api/system/health
系统健康	模块状态、活跃降级、冷却截止与最近告警	docs/screenshots/10-system-health.png、/api/system/health
术语语义字典	知识图谱页：14 张术语卡，已覆盖的标出关联文档数并给出语料原文定义句（带 chunk id 与字符偏移），未覆盖的如实标注而不是编造释义	docs/screenshots/11-term-dictionary.png、/api/knowledge/terms

模块	展示方式	证据文件
术语 → 文档两跳	知识图谱页：术语节点与文档节点分型渲染的两跳关系	docs/screenshots/11-term-dictionary.png、/api/knowledge/graph
模型降级链与角色绑定	赛题指标页：三级降级链（含每一级被跳过的原因）与七个角色的模型绑定	docs/screenshots/12-model-chain.png、/api/system/model-chain
推荐问法	问答区：题目直接取自固定评测集，点一下即填入输入框；拒答类题目不入列	docs/screenshots/13-question-suggestions.png、/api/qa/suggestions
技术详情	回答展开后的本轮计划、工具调用与返回条数、耗时与已知限制	docs/screenshots/14-technical-detail.png
富化	事件详情中的 CVSS、受影响区间、修复版本与来源标识	docs/CASE_ENRICHMENT_INFERENCE.md
问答与多轮	多轮追问、跨文档引用、拒答三组界面截图	docs/screenshots/b-qa-0*.png
处置闭环	高优先级队列、复测关闭与操作前后对比截图	docs/A_DISPOSITION_LIVE_CLOSURE.md

上表前两行是浏览器级实测，不是结构性断言：tools/browser_check.py 用真实 Chromium 打开页面，把渲染出来的行数与 /api/system/health 的返回逐项对齐——来源方向 20 行对 20 个来源、系统健康 4 行对 4 个模块、徽标文案与模块状态计数一致、每个来源的状态文案都能在接口取值里找到对应项，截图分别是 docs/screenshots/09-source-health.png 与 10-system-health.png。这两张图的数据全部来自 source_state 与 source_degradations 的持久化记录，前端不做任何推算。

十、性能测试结果与分析

指标	实测	说明
监测时效性（混合，见附录 B）	p50 = 15.69 小时；6 小时内 31.8%	296 个有效样本；赛题合格档为 ≤24 小时
监测时效性（漏洞类，赛题口径）	GHSA 中位 1.8 小时；NVD 中位 12.4 小时	赛题问的是漏洞信息从发布到采集的延迟，故按来源类别分列；样本 67 / 119，取自 docs/A_TIMELINESS_ATTRIBUTION.md 第 3 节的交付机读数
问答引用命中	0.9722（35/36）	同一批题目在修复前的提交上重跑为 0.8889，差异可归因于代码
问答响应	分位数按批次独立统计	赛题目标单题 5 秒内
富化关系准确率	Precision 1.0 / Recall 0.734	抽样穷尽金标准，94 条可判定样本
连续运行	累计 20 个采集日，定时 10 天	7 天连续运行目标：当前窗口未达成——计划任务窗口 10-11 到期后 10-12 无计划运行，连续天数已重新起算，恢复运行后需重新累积
自动化测试	701 passed, 0 skipped, 0 failed	交付机全量 pytest；语料与衍生副本均已交付核验

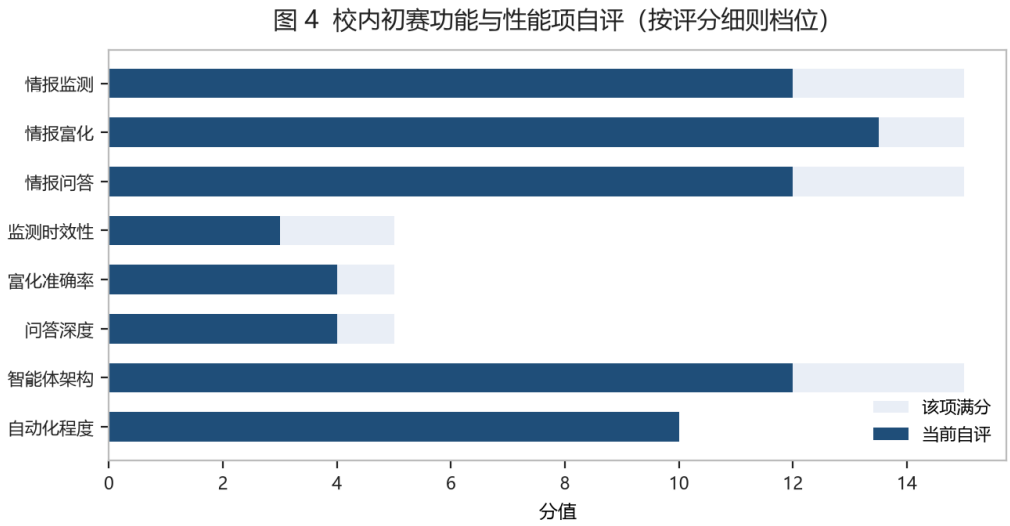


图 4 校内初赛功能与性能项自评（按评分细则档位）

对时效性的归因：混合口径的中位延迟是 18.1 小时，但如果就此说“采集慢”是错的。按来源类别拆开看，延迟最大的一段根本不在漏洞类——arxiv 36.5 小时、openalex 61.8 小时，它们衡量的是论文与索引的发布节奏；而赛题问的是漏洞信息。漏洞类里 GHSA 中位 1.8 小时、NVD 中位 12.4 小时。

瓶颈在上游而不在我们：采集间隔是 1 小时（计划任务每小时触发一次），而同一个 CVE 在 MITRE 登记记录与NVD 记录之间的发布时差本身就有中位 0.68 小时（79 对样本）。也就是说上游两个登记库是先后滚动的，我们把 NVD 那条记到时，NVD 自己才刚发出来。跨来源时间线的可回读样例（含事件 ID 与两个来源时间戳）见 docs/A_TIMELINESS_ATTRIBUTION.md 第 4 节，机读产物是 reports/timeliness-attribution.json。

因此结论写成一句可引用的话：采集间隔 1 小时；GHSA 中位 1.8 小时，NVD 中位 12.4 小时，差值来自上游发布与入库节奏。分列把混合口径里那些与漏洞无关的发布节奏剥出去，但混合口径本身保留在附录 B，两个数都能被追问。

十一、创新点总结

- 证据分层而非结论合并：同一事件的多个来源区间分别保留，避免合并后给出错误的修复版本。
- 降级而不是重试到底：连续失败触发降频或切换备用源，并且恢复后自动回到采集集合。
- 口径与展示同源：页面、接口与证据文件读同一份持久化状态，指标不靠前端估算。
- 可复现的工程证据：容器验收、CI、回滚、自愈演练都留下可重跑的脚本与产物。

附录 B：时效性的混合口径（保留，不进正表）

第十章的主表按来源类别分列，因为赛题问的是漏洞信息。混合口径不删除、不隐藏，列在这里备查：267 个有效样本，中位 18.1 小时，6 小时内 27.7%、12 小时内 39.7%、24 小时内 58.4%。这一档只够赛题的合格档，分列并不改变档位判定——本项交付的是归因，不是抬分。

口径	来源	样本	中位延迟	最大延迟
漏洞类	ghsa	67	1.8 小时	94.1 小时
漏洞类	nvd	119	12.4 小时	287.3 小时
学术类	arxiv	76	36.5 小时	91.8 小时
学术类	openalex	4	61.8 小时	61.8 小时
其他	anquanke	1	89.9 小时	89.9 小时
混合（本表合计）	全部来源	267	18.1 小时	—

类别归属取自 app/sources.py 的 category 字段：漏洞类含 cve_database、ecosystem_advisory、community_advisory、exploitation_intel、vendor_advisory 与 security_blog；学术类为 academic_paper；政策类为 policy_regulation、technical_standard、risk_framework、threat_framework。不另造一套分类，避免出现“报告里的类别”与“代码里的类别”两套口径。

附：评分细则档位对照

每一条评分细则对应到合格／良好／优秀的哪一档、证据在哪，见 docs/COMPETITION_RUBRIC_MAPPING.md。该表同时列出三处必须如实说明的缺口：无实时监测、时效性只到合格档、跨文档路径 10/21。